At the core of the MCP is a standardized client–server protocol that defines how web applications, AI assistants, or other systems can connect to external tools and data sources through LLMs. An MCP client handles communication with one or more MCP servers, which expose tools and resources in a consistent, structured format. While an AI agent may use an MCP client as part of its architecture, the two are conceptually distinct, as the client enables protocol-level communication, while the agent handles reasoning and decision making. This architecture supports bidirectional data flow, which allows applications to send requests to tools and receive structured responses that can be incorporated into the AI's reasoning and outputs.

In practice, this means an AI application, whether a chat interface, an IDE, or another AI-powered tool, can interact seamlessly with multiple data sources, such as databases, filesystems, APIs, and productivity tools. The MCP server ensures that these interactions follow a standardized protocol, allowing developers to reuse tools across different AI applications without rewriting code.

## 12.3 Connecting Next.js and the Vercel AI SDK with the MCP

While the Vercel AI SDK includes experimental support for setting up an MCP server, you aren't limited to that approach. A web app like Astra can act as an *MCP client,* directly connecting to one or more MCP servers to request and process data, or even as an *MCP server* itself, exposing its own tools for use by other agents or applications.

Let's see how we can integrate the Vercel AI SDK to build a simple web application that fetches fun facts using an external API. In this example, we'll use the Chuck Norris Jokes API (https://api.chucknorris.io), a free JSON API that provides random jokes and facts.

You can easily adapt this example to other free public APIs available at https://publicapi.dev/, such as trivia, quotes, and weather services. The same integration pattern applies: the MCP server defines a tool that retrieves data from an external API, and the web application uses that tool to deliver dynamic responses to user prompts.

### Project code: vercel-ai-sdk-mcp

The code for this example project can be found in the ch12/vercel-ai-sdk-mcp folder. You can use that example by running this command in the root folder of the project repository:

```
$ npm run dev -w ch12/vercel-ai-sdk-mcp
```

You'll need the Gemini API key or OpenAI key for this to work as the application uses services from different vendors. These are configured as environment variables:

```
// .env
GOOGLE_API_KEY=<GEMINI_API_SECRET_KEY>
OPENAI_API_KEY=<OPEN_AI_SECRET_KEY>
```